

Primitives, Precedence & Scalars

Primitives

What *is* a primitive?

A primitive is a value your language treats as **built-in and indivisible**

The language doesn't let you look inside it or ask it for methods on its own data (in most languages).

The following code are some of the primitives defined in `Java`

```
1 // Java
2 int age = 21;
3 double gpa = 3.7;
4 boolean isEnrolled = true;
5 char grade = 'A';
```

Contrast this with **objects/composites**

- a `String`, a `List`, a `struct` all bundle primitives together.

Type systems

Static vs. dynamic - is it type checked before running (compile time) or while running?

Strong vs. weak - does the language let types silently convert into each other?

Same expression, three outcomes:

```
1 // JavaScript (dynamic, weak)
2 "5" + 3
```

```
1 # Python (dynamic, strong)
2 "5" + 3
```

```
1 // Java (static, strong)
2 int x = "5" + 3;
```

Static typing

Types are checked *before* the program runs, at **compile time**.

Every variable's type is fixed the moment it's declared, and it **cannot change** later.

```
1 // Java
2 String name = "Ana";
3 name = 42;           // compile error: String ≠ int
```

Because the compiler knows every type up front, it **catches** mismatches before the program ever starts.

- **Examples:** Java , C , C++ , Rust , Go
- The *cost*: you need to commit to a type from the very start.

Dynamic typing

Types are checked *while* the program runs, at *runtime*.

A variable has no fixed type - its type is whatever value it *currently* holds.

```
1  # Python
2  x = 5          # x is an int
3  x = "five"     # now x is a string - no error
```

Type errors only surface when the offending line actually *executes*.

```
1  # Python
2  name = "Ana"
3  name + 5       # TypeError - only when this line runs
```

- **Examples:** Python , JavaScript , Ruby
- The *gain*: less ceremony
- the *cost*: mistakes hide until they happen.

Strong typing

The language does **not** let types silently convert into each other.

Mismatched types are an *error*, you **must** convert explicitly.

```
1  # Python
2  "5" + 3          # TypeError: can only concatenate str (not "int") to str
```

Explicit conversion is the fix:

```
1  # Python
2  int("5") + 3     # 8
```

- **Examples:** Python , Java , Rust
- Strong typing *refuses* to guess what you meant.

Weak typing

The language *silently converts* types to make an operation work.

Convenient in the moment, the result may *not* be what you meant.

```
1 // JavaScript
2 "5" + 3      // "53" - + prefers string concatenation
3 "5" - 3      // 2    - - has no string meaning, so it converts
```

- **Examples:** JavaScript, C
- That `"5" + 3` is the *same expression* as earlier - one line, three outcomes, three languages.

Floating points

```
1 // javascript
2 0.1 + 0.2 === 0.3
3 0.1 + 0.2
```

Floats are a **finite binary approximation** of an infinite decimal space

"Incorrect" outputs of floating point operations are a bug,

they are a rounding trade-off (IEEE 754).

Some fractions just can't be represented *exactly* in binary, the same way $1/3$ can't be represented exactly in decimal.

Rule of thumb: never compare floats with `==` Compare with a tolerance:

```
1 # Python
2 abs(x - 0.3) < 1e-9
```

Booleans

A boolean is a value that is either `true` or `false` — nothing else.

Named after **George Boole** (1815–1864), who showed that logic could be written down as *algebra*.

```
1 # Python
2 is_enrolled = True
```

It's **slightly different** from every other primitive scalar type

- every other primitive has a *range* of possible values
- a boolean has exactly *two* possible values, there is no *third* state

Every comparison

- all the **result of a comparison** become booleans - `5 > 3` returns a `boolean`, not an `int` or a `string`
- it's the **only** type that drives **control flow**: `if`, `while`, and `for` all ask for a boolean

Truthiness

Booleans are only `true` or `false` - but many languages let *non-boolean* values be used **wherever a boolean is expected**.

When a value gets converted to a boolean automatically, we call its answer *truthy* or *falsy*.

- a **truthy** value acts like `true` when placed in an `if` or a condition
- a **falsy** value acts like `false`

```
1 // javascript
2 if ("hello") { ... } // "hello" is truthy - this runs
3 if (0) { ... }       // 0 is falsy - this doesn't
```

Boolean as a primitive - truthy/falsy varies

Value	JavaScript	Python	Java
<code>0</code>	falsy	falsy	<i>n/a - not a boolean</i>
<code>" "</code> (empty string)	falsy	falsy	<i>n/a</i>
<code>[]</code> (empty array)	truthy	falsy	<i>n/a</i>
<code>null</code> / <code>None</code>	falsy	falsy	<i>n/a - NullPointerException risk</i>
<code>"0"</code> (string zero)	truthy	truthy	<i>n/a</i>

Java refuses to play this game at all

`if (5)` won't even compile.

That's a **design choice**, not a missing feature.

Analogies

Int vs. float - whole apples vs. apple sauce.

- One is discrete and countable, the other is blended and approximate.

A variable is a labeled shoebox.

- The primitive type tells you *what shape of box* it is
- an `int` box only fits whole numbers,
- a `boolean` box only fits true/false.

Common early mistakes

1. Assuming integers are *infinite*

Remember that computers are ones and zeroes, and computers have a **limited** amount of them

So an integer **isn't** infinite*, it's bound by how many 1s and 0s are assigned to them

* Python actually is infinite, since it automatically increases the amount of bits a number has

But a language like `c` will stop at 2147483647 and loop back to -2147483647

Common early mistakes

2. Assuming floats are *exact* -> `if (x == 0.3)` will never fire

```
Try 0.1 + 0.2
```

Binary can only represent fractions that are a sum of negative powers of two ($1/2, 1/4, 1/8, \dots$) exactly.

A "nice" decimal like 0.1 doesn't terminate in binary ($0.1_{10} = 0.0001100110011\dots_2$),

$1/3$ never terminates in decimal. IEEE 754 stores the closest 53-bit approximation (a `double`), so `=` against the expected "nice" value quietly fails.

Common early mistakes

3. Confusing `=` (assignment) with `==` (comparison)

In c

```
1 // C
2 if (x = 5) { ... } // assigns 5, then evaluates truthy – always runs!
```

Why this happens:

- in C-family languages `=` is an *expression*
- it assigns the value **and** evaluates to it, so `if (x = 5)` runs the body for *any* non-zero value and silently destroys the original `x`.
- A comparison returns a boolean, an assignment returns the assigned value.

In Python the equivalent `=`-in-condition is a syntax error, not a silent bug:

```
1 # Python
2 if x = 5:      # SyntaxError: invalid syntax
```


Precedence

PEMDAS, extended

You already know: **P**arens > **E**xponents > **M**ultiply/**D**ivide > **A**dd/**S**ubtract.

Programming languages extend that ladder downward:

```
1  # Pseudocode
2  ()                highest
3  ** (exponent)
4  * / %
5  + -
6  < > ≤ ≥ = ≠
7  && (and)
8  || (or)
9  = += -= ...      lowest
```

```
1  # Python
2  2 + 3 * 4          # 14, not 20
3  2 + 3 == 5 and 1   # comparisons bind tighter than 'and'
```

Associativity

Precedence tells you *what* goes first.

Associativity tells you *which way* to read operators of the *same* precedence.

```
1  # Python
2  2 ** 3 ** 2
3  # ** is right-associative: 2 ** (3 ** 2) = 2 ** 9 = 512
```

```
1  # Python
2  10 - 3 - 2
3  # - is left-associative: (10 - 3) - 2 = 5
```

Precedence traps

```
1 # Python
2 True or False and False
```

More "what does this print?"

```
1 # Python
2 0 and 5
3 "a" or "b"
4 1 < 2 < 3
5 2 ** 3 ** 2
6 (not 0) and 1
```

Just use parentheses

Knowing the rules matters, but you should know when to stop relying on them for readability:

```
1 // JavaScript
2 // Technically correct, precedence-perfect:
3 if (a && b || c && !d)
4
5 // Clearer for the next human who reads it:
6 if ((a && b) || (c && !d))
```

Exercise

```
1  # Python
2  x = 5
3  y = 2
4  print(x + y * 2 > 8 and not y == 2)
```

Exercise

```
1  # Python
2  x = 5
3  y = 2
4  print(x + y * 2 > 8 and not y == 2)
```

Trace:

1. $y * 2 \rightarrow 4$
2. $x + 4 \rightarrow 9$
3. $9 > 8 \rightarrow \text{True}$
4. $y == 2 \rightarrow \text{True}$
5. $\text{not True} \rightarrow \text{False}$
6. $\text{True and False} \rightarrow \text{False}$

Scalars

Scalar vs. composite

A **scalar** is a single atomic value - one number, one boolean, one character.

A **composite** (array, list, struct, object) bundles multiple values together.

```
1  # Pseudocode
2  scalar:    42
3  composite: [42, 7, 13]      ← an array of three scalars
4  composite: {name: "Ana", age: 21} ← a struct/object of two scalars
```

Every primitive from the first section was a scalar.

Not every scalar has to be a language *"primitive"* though

Some languages have scalar types that aren't primitives (e.g. a `Date` object treated as one logical value).

Arrays are collections of scalars

```
1  # Python
2  totals = [12, 45, 7]
3  totals[1]      # 45 – indexing INTO a composite gets you back to a scalar
```

We'll talk about these later but it's useful to know

Exercise

1. Predict the output, then explain in one sentence *why*:

```
1 # Python
2 print(2 + 3 * 2 ** 2)
```

2. True or false, and why: *"An array is a primitive type."*

3. Name one language where `"5" + 3` throws an error, and one where it doesn't.